

Name: - Shubham Das

S. A. P I. D: - 590033500

Date: - 01/09/2026

Course: - B.TECH (CSE)

Question 1: - Write a C program to find the volume, lateral surface area, and total surface area of a square pyramid when its base side, slant height, and height are given.

Part 1: - Thorough understanding of the problem statement

Input: - 'a' (Base side length, Variable type: 'float', Unit: meters or 'm')

'l' (Slant height, Variable type: 'float', Unit: meters or 'm')

'h' (Vertical height, Variable type: 'float', Unit: meters or 'm')

Output:- 'volume' (Volume, Variable type: 'float', Unit: cubic meters or m^3)

'lsa' (Lateral Surface Area, Variable type: 'float', Unit: square meters or m^2)

'tsa' (Total Surface Area, Variable type: 'float', Unit: square meters or m^2)

Formula: - Volume: $\text{volume} = (1.0 / 3.0) * a * a * h$;

Lateral Surface Area: $\text{lsa} = 2.0 * a * l$;

Total Surface Area: $\text{tsa} = (a * a) + \text{lsa}$;

Part 2: - Writing and evaluating the algorithm

Step 1: - Start

Step 2 : - Declare floating-point variables 'a', 'l', 'h', 'lsa', 'tsa'.

Step 3: - Prompt the user to enter the base side ('a').

Step 4: - Read the input into the 'a' variable.

Step 5: - Prompt the user to enter the slant height ('l').

Step 6: - Read the input into the 'l' variable.

Step 7: - Prompt the user to enter the vertical height ('h').

Step 8: - Read the input into the 'h' variable.

Step 9: - Check if 'a', 'l', or 'h' are less than or equal to '0'. If true, print an error message (dimensions must be positive) and terminate the programme.

Step 10: - Calculate Volume = $(1.0 / 3.0) * a * a * h$.

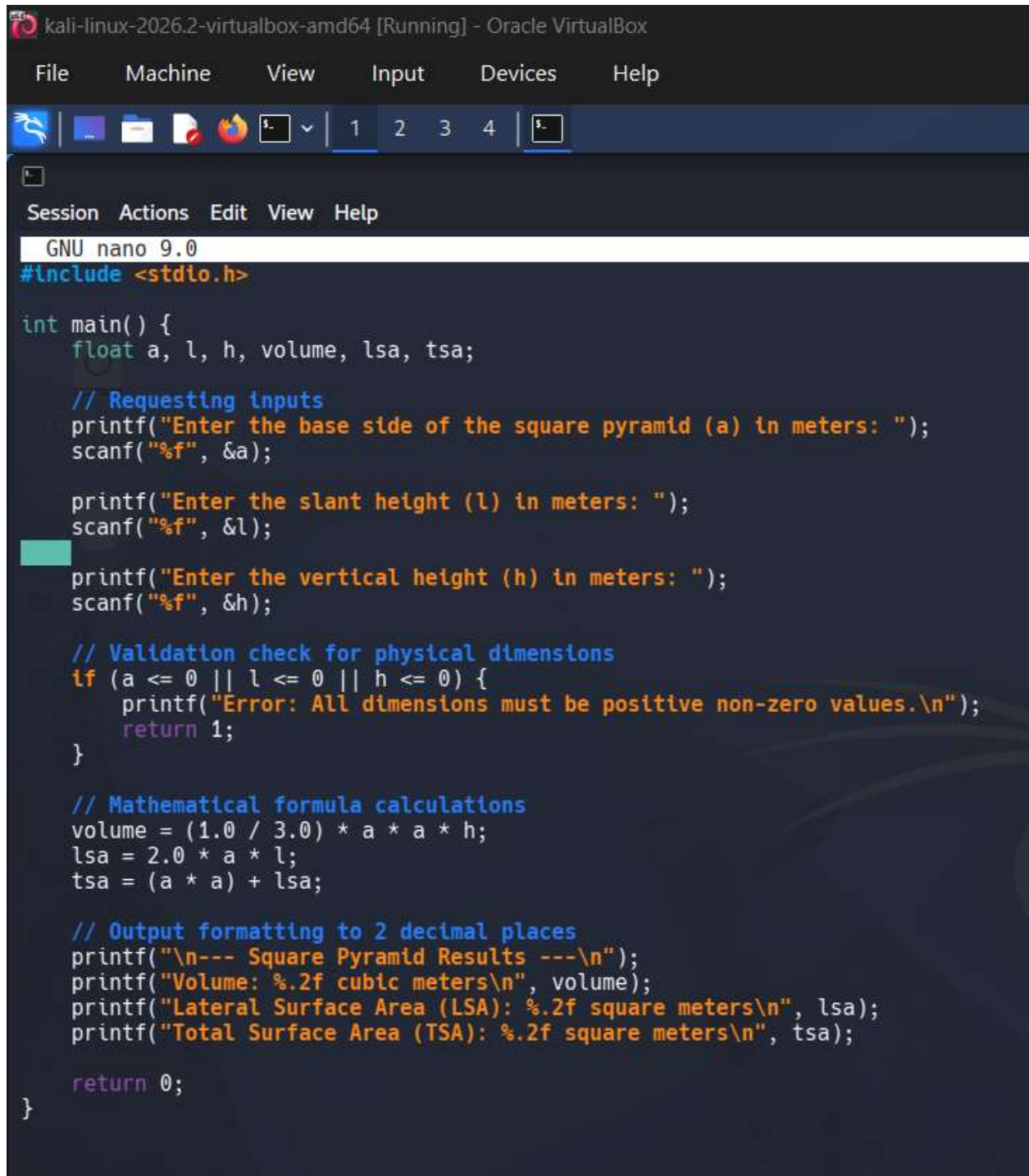
Step 11: - Calculate LSA = $2.0 * a * l$.

Step 12: - Calculate TSA = $(a * a) + \text{lsa}$.

Step 13: - Display 'volume', 'lsa', and 'tsa' rounded to two decimal places.

Step 15: - Stop

Part 3:- Writing and compiling the C programme



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

GNU nano 9.0
#include <stdio.h>

int main() {
    float a, l, h, volume, lsa, tsa;

    // Requesting inputs
    printf("Enter the base side of the square pyramid (a) in meters: ");
    scanf("%f", &a);

    printf("Enter the slant height (l) in meters: ");
    scanf("%f", &l);

    printf("Enter the vertical height (h) in meters: ");
    scanf("%f", &h);

    // Validation check for physical dimensions
    if (a <= 0 || l <= 0 || h <= 0) {
        printf("Error: All dimensions must be positive non-zero values.\n");
        return 1;
    }

    // Mathematical formula calculations
    volume = (1.0 / 3.0) * a * a * h;
    lsa = 2.0 * a * l;
    tsa = (a * a) + lsa;

    // Output formatting to 2 decimal places
    printf("\n--- Square Pyramid Results ---\n");
    printf("Volume: %.2f cubic meters\n", volume);
    printf("Lateral Surface Area (LSA): %.2f square meters\n", lsa);
    printf("Total Surface Area (TSA): %.2f square meters\n", tsa);

    return 0;
}
```

Part 4 : Running and executing the algorithm on a platform (Kali Linux VM)

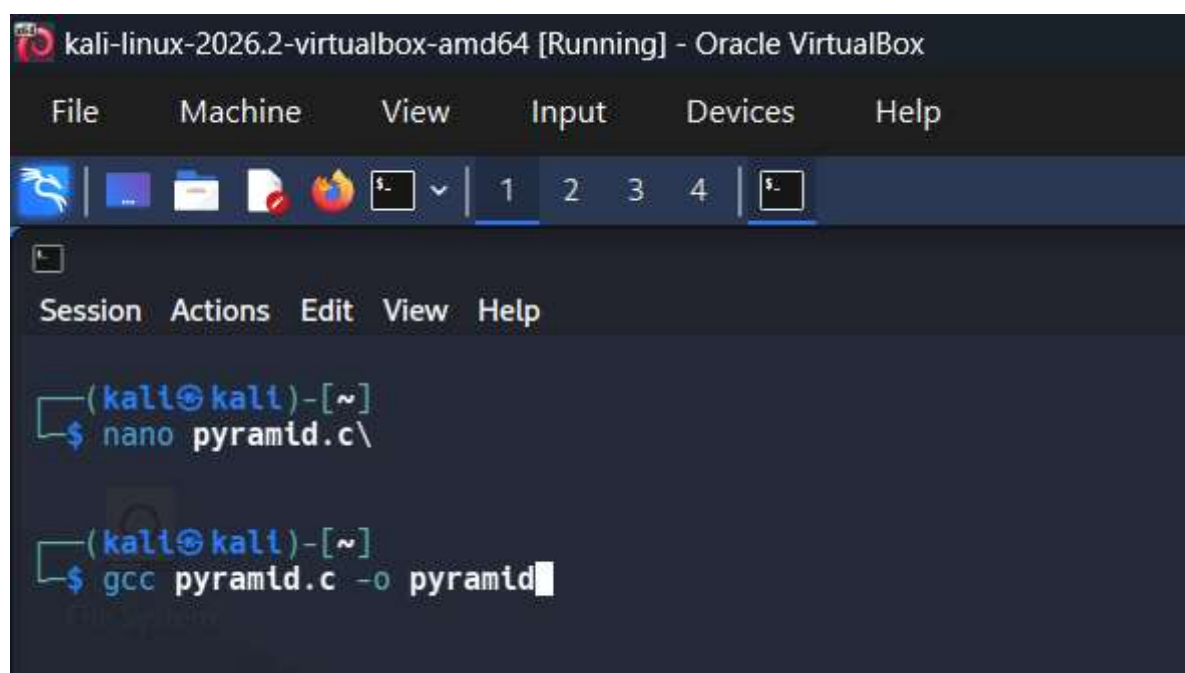
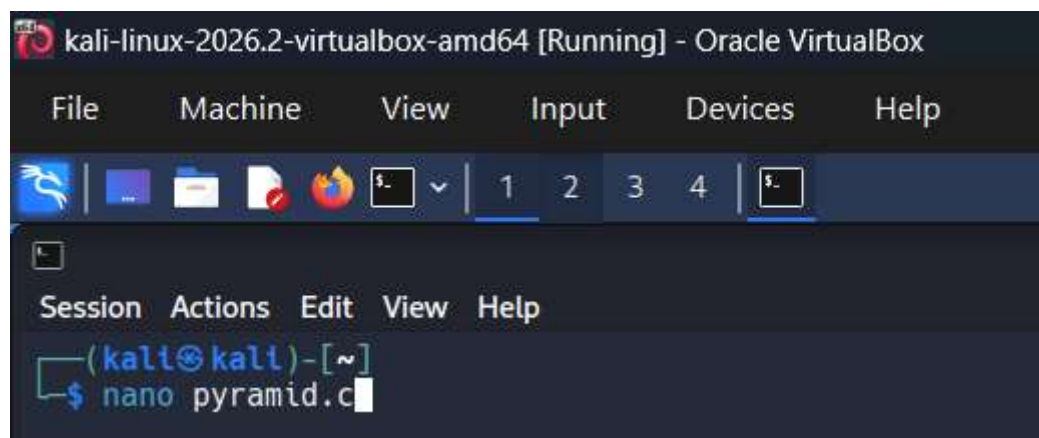
Step 1: - Open the Terminal

Step 2: - Create the Source File

Step 3: - Write and Save Code

Step 4: - Compile the programme

Step 5: - Run the Binary Executable programme



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

(kali@kali)-[~]
$ ./pyramid
Enter the base side of the square pyramid (a) in meters: 10
Enter the slant height (l) in meters: 13
Enter the vertical height (h) in meters: 12

--- Square Pyramid Results ---
Volume: 400.00 cubic meters
Lateral Surface Area (LSA): 260.00 square meters
Total Surface Area (TSA): 360.00 square meters

(kali@kali)-[~]
$ ./pyramid
Enter the base side of the square pyramid (a) in meters: 0
Enter the slant height (l) in meters: 5
Enter the vertical height (h) in meters: 4
Error: All dimensions must be positive non-zero values.

(kali@kali)-[~]
$
```

Part 5: - Compiling, executing and evaluating against all the test cases

TEST CASE	INPUT(S)	EXPECTED OUTPUT(S)	ACTUAL OUTPUT(S)	RESULT (PASS/FAIL)
1	a = 6 l = 5 h = 4	Vol: 48.00 LSA: 60.00 TSA: 96.00	Vol: 48.00 LSA: 60.00 TSA: 96.00	PASS
2	a = 10 l = 13 h = 12	Vol: 400.00 LSA: 260.00 TSA: 360.00	Vol: 400.00 LSA: 260.00 TSA: 360.00	PASS
3	a = 0 l = 5 h = 4	Error: must be positive values	Error: must be positive values	PASS

Part 6: - Evaluation

The algorithm strictly executes in constant time, utilizing only basic scalar variables and direct arithmetic operators.

Question 2: - Write a C programme to find each angle when two supplementary angles are in a given ratio.

Part 1: - Thorough understanding of the problem statement

Input: - r1 (First part of the ratio, Variable type: 'float', Unit: unitless ratio)
r2 (Second part of the ratio, Variable type: 'float', Unit: unitless ratio)

Output: - 'angle1' (First supplementary angle, Variable type: 'float', Unit: degrees or °)
'angle2' (Second supplementary angle, Variable type: 'float', Unit: degrees or °)

Formula: - Supplementary angles sum strictly to 180 degrees
'Total_ratio = r1 + r2'
'angle1 = (r1 / total_ratio) * 180.0';
'angle2 = (r2 / total_ratio) * 180.0';

Part 2: - Writing and evaluating the algorithm

Step 1: - Start

Step 2: - Declare floating-point variables 'r1', 'r2', 'total_ratio', 'angle1' and 'angle2'.

Step 3: - Prompt the user to enter the first ratio value ('r1').

Step 4: - Read the input into the 'r1' variable.

Step 5: - Prompt the user to enter the second ratio value ('r2').

Step 6: - Read the input into the 'r2' variable.

Step 7: - Validate the inputs. If 'r1 <= 0' or 'r2 <= 0', display an error message (ratios must be positive) and terminate the programme to prevent division by zero or negative angles.

Step 8: - Calculate 'total_ratio = r1 + r2'.

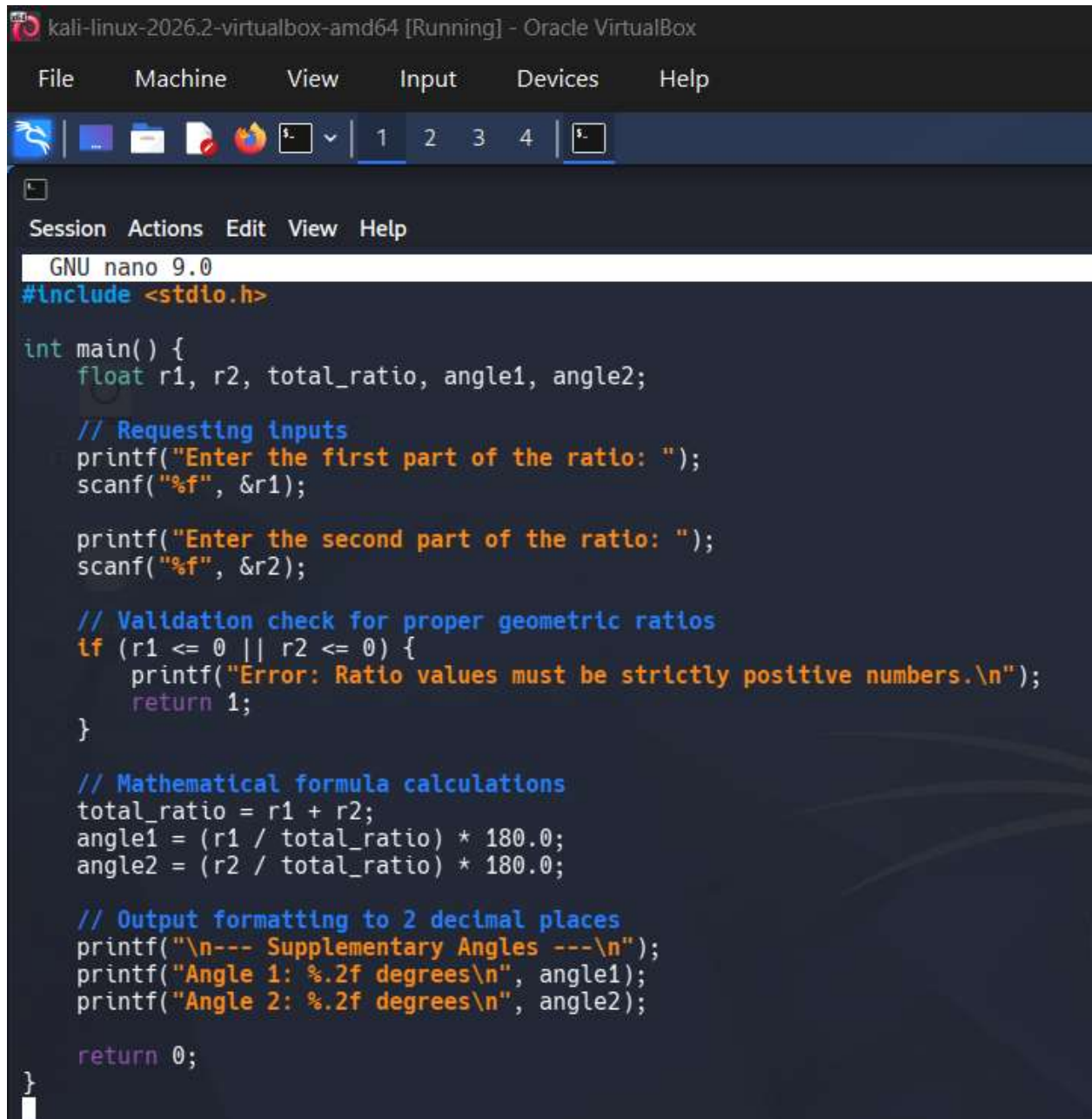
Step 9: - Calculate 'angle1 = (r1 / total_ratio) * 180.0'.

Step 10: - Calculate 'angle2 = (r2 / total_ratio) * 180.0'.

Step 11: - Display the calculated values for 'angle1' and 'angle2'.

Step 12: - Stop

Step 3: - Writing and compiling the C programme.



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
Session Actions Edit View Help
GNU nano 9.0
#include <stdio.h>

int main() {
    float r1, r2, total_ratio, angle1, angle2;

    // Requesting inputs
    printf("Enter the first part of the ratio: ");
    scanf("%f", &r1);

    printf("Enter the second part of the ratio: ");
    scanf("%f", &r2);

    // Validation check for proper geometric ratios
    if (r1 <= 0 || r2 <= 0) {
        printf("Error: Ratio values must be strictly positive numbers.\n");
        return 1;
    }

    // Mathematical formula calculations
    total_ratio = r1 + r2;
    angle1 = (r1 / total_ratio) * 180.0;
    angle2 = (r2 / total_ratio) * 180.0;

    // Output formatting to 2 decimal places
    printf("\n--- Supplementary Angles ---\n");
    printf("Angle 1: %.2f degrees\n", angle1);
    printf("Angle 2: %.2f degrees\n", angle2);

    return 0;
}
```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

Step 1: - Open the Terminal

Step 2: - Create the Source File

Step 3: - Write and Save Code

Step 4: - Compile the programme

Step 5: - Run the Binary Executable programme

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

(kali@kali)-[~]
$ nano angles.c

(kali@kali)-[~]
$ gcc angles.c -o angles

(kali@kali)-[~]
$ ./angles
Enter the first part of the ratio: 2
Enter the second part of the ratio: 3

--- Supplementary Angles ---
Angle 1: 72.00 degrees
Angle 2: 108.00 degrees

(kali@kali)-[~]
$ ./angles
Enter the first part of the ratio: 1
Enter the second part of the ratio: 1

--- Supplementary Angles ---
Angle 1: 90.00 degrees
Angle 2: 90.00 degrees

(kali@kali)-[~]
$ ./angles
Enter the first part of the ratio: 5
Enter the second part of the ratio: -2
Error: Ratio values must be strictly positive numbers.

(kali@kali)-[~]
$
```

Part 5: - Compiling, executing and evaluating against all the test cases

TEST CASE	INPUT(S)	EXPECTED OUTPUT(S)	ACTUAL OUTPUT(S)	RESULT (PASS/FAIL)
1	r1 = 1 r2 = 1	Angle 1: 90.00 Angle 2: 90.00	Angle 1: 90.00 Angle 2: 90.00	PASS
2	r1 = 2 r2 = 3	Angle 1: 72.00 Angle 2: 108.00	Angle 1: 72.00 Angle 2: 108.00	PASS
3	r1 = 5 r2 = -2	Error: values must be strictly positive	Error: values must be strictly positive	PASS

Part 6: - Evaluation

The algorithm executes in constant time. It utilizes fundamental arithmetic operations mapping fractional ratios against a static scalar (180.0).